

Wydział: Fizyki i Informatyki Stosowanej	Autor/Autorzy: Maciej Szeptuch	Rok: 2026	Grupa: 1	Zespół: -	
PRACOWNIA FIZYCZNA AGH	WFiS	Równanie Poissona: minimalizacja, optymalizacja gradientowa i Monte Carlo	Ćwiczenie nr: 5		
Data wykonania: 2026-05-22	Data oddania: 2026-05-22	Zwrot do popr.:	Data oddania:	Data zaliczenia:	Ocena:

1 Cel ćwiczenia

Celem ćwiczenia było rozwiązanie równania Poissona w dwuwymiarowej, uziemionej na brzegach przestrzeni. W niniejszym ćwiczeniu wykorzystano różne metody numeryczne rozwiązania:

- minimalizacji funkcjonału energii,
- optymalizacji gradientowej,
- metody Monte Carlo.

2 Aparatura i metodyka wykonania

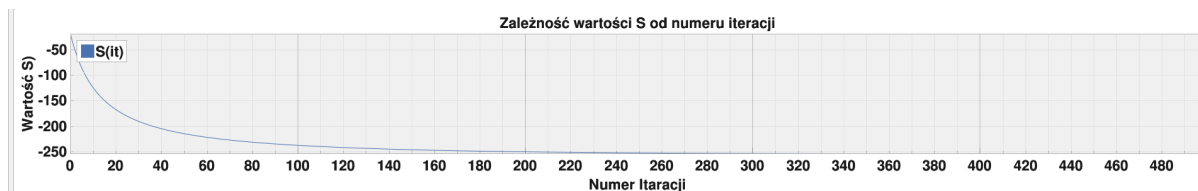
W ramach eksperymentu (Prove of concept) kod do symulacji i prezentacji wyników wykonano w języku Go. Do prezentacji wykorzystano bibliotekę `giu` (framework `Dear ImGui`).

3 Wyniki pomiarów

3.1 Minimalizacja bezpośrednia

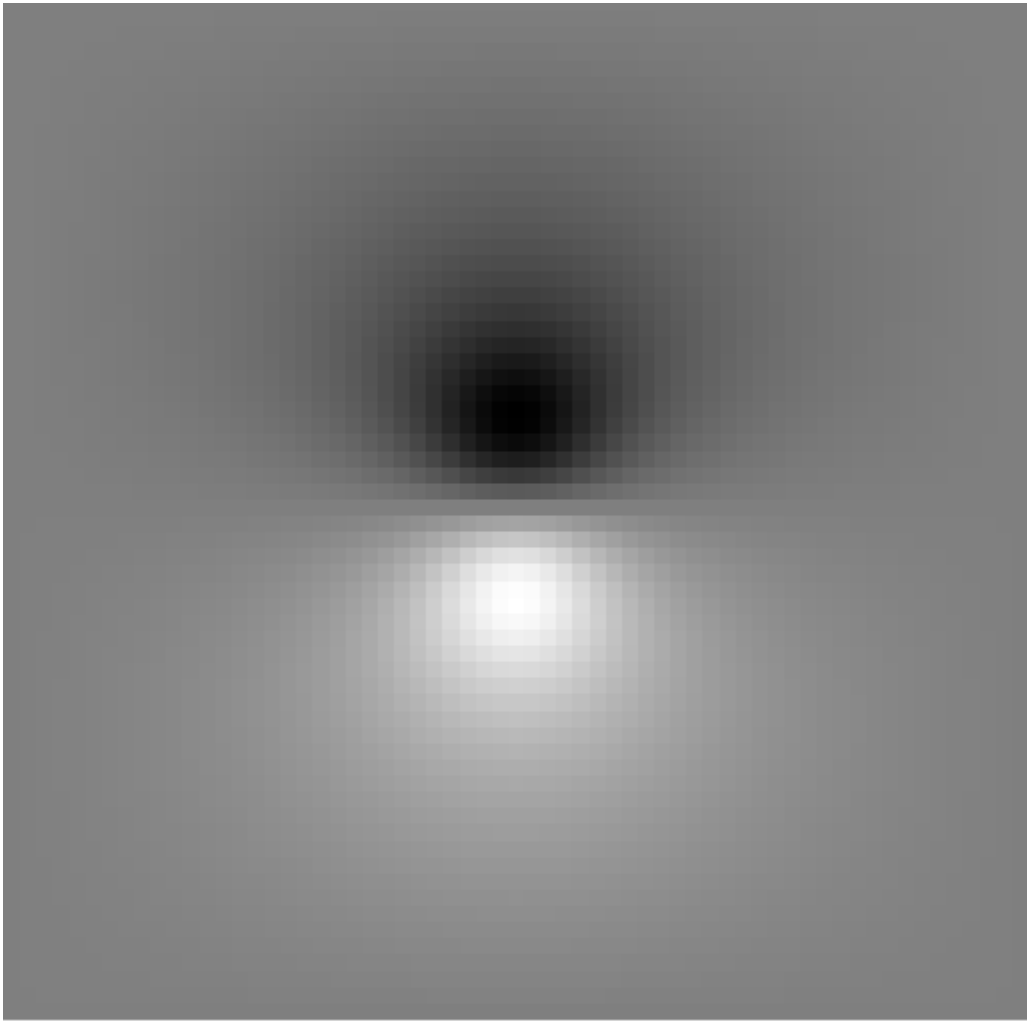
Najpierw wykonano minimalizację bezpośrednio wartości S_{loc} . W tym celu w każdej iteracji zmieniano wartość $u(i, j)$ o jedną z wartości $\delta \in \left\{ 0, 0.5, 1, \frac{1}{4} \frac{3S_1 - 4S_2 + S_3}{S_1 - 2S_2 + S_3} \right\}$ tak, aby uzyskać jak najmniejszą wartość S_i (gdzie $S_i = S(\delta_i)$).

Uzyskano następującą zależność wartości S od numeru iteracji:



Rys. 3.1: Zależność S od numeru iteracji dla minimalizacji bezpośredniej S .

Po ostatniej iteracji otrzymano następujący rozkład wartości $u(i, j)$

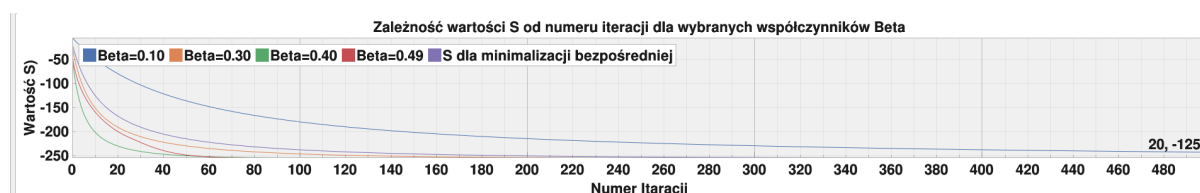


Rys. 3.2: Rozkład $u(i, j)$ po minimalizacji bezpośredniej S.

3.2 Minimalizacja gradientowa

Można również dokonać minimalizacji S poprzez wyliczenie wartości gradientu. Dzięki temu zamiast czterokrotnie liczyć wartości S można to zrobić tylko 2 razy. Przyjmuje się wartość $d = 0.001$ i wylicza się wartość S dla $u(i, j) = u(i, j) \pm d$. Następnie wyznacza się wartość $\nabla S = \frac{S_+ - S_-}{2d}$ i zmniejsza się wartość $u(i, j)$ o $\beta \nabla S$. Obliczenia wykonano dla 4 różnych wartości współczynnika $\beta \in \{0.1, 0.2, 0.3, 0.4, 0.49\}$.

Na poniższym wykresie zestawiono zależności S od numeru iteracji dla różnych wartości β oraz minimalizacji bezpośredniej.



Rys. 3.3: Zależność S od numeru iteracji dla minimalizacji bezpośredniej oraz gradientowej dla różnych wartości β oraz dla minimalizacji bezpośredniej.

Dla $\beta \rightarrow 0.5$ operacja jest szybciej zbieżna. Można zaobserwować, że pomimo, że $\beta = 0.49$ teoretycznie jest zbiega się wolniej, wartość S będzie ostatecznie mniejsza niż dla $\beta = 0.4$.

Poniżej przedstawiono rozkład $u(i, j)$ dla $\beta = 0.1$ oraz $\beta = 0.49$. Nie zamieszczono pozostałych rozkładów, ponieważ są one praktycznie identyczne jak $\beta = 0.49$.

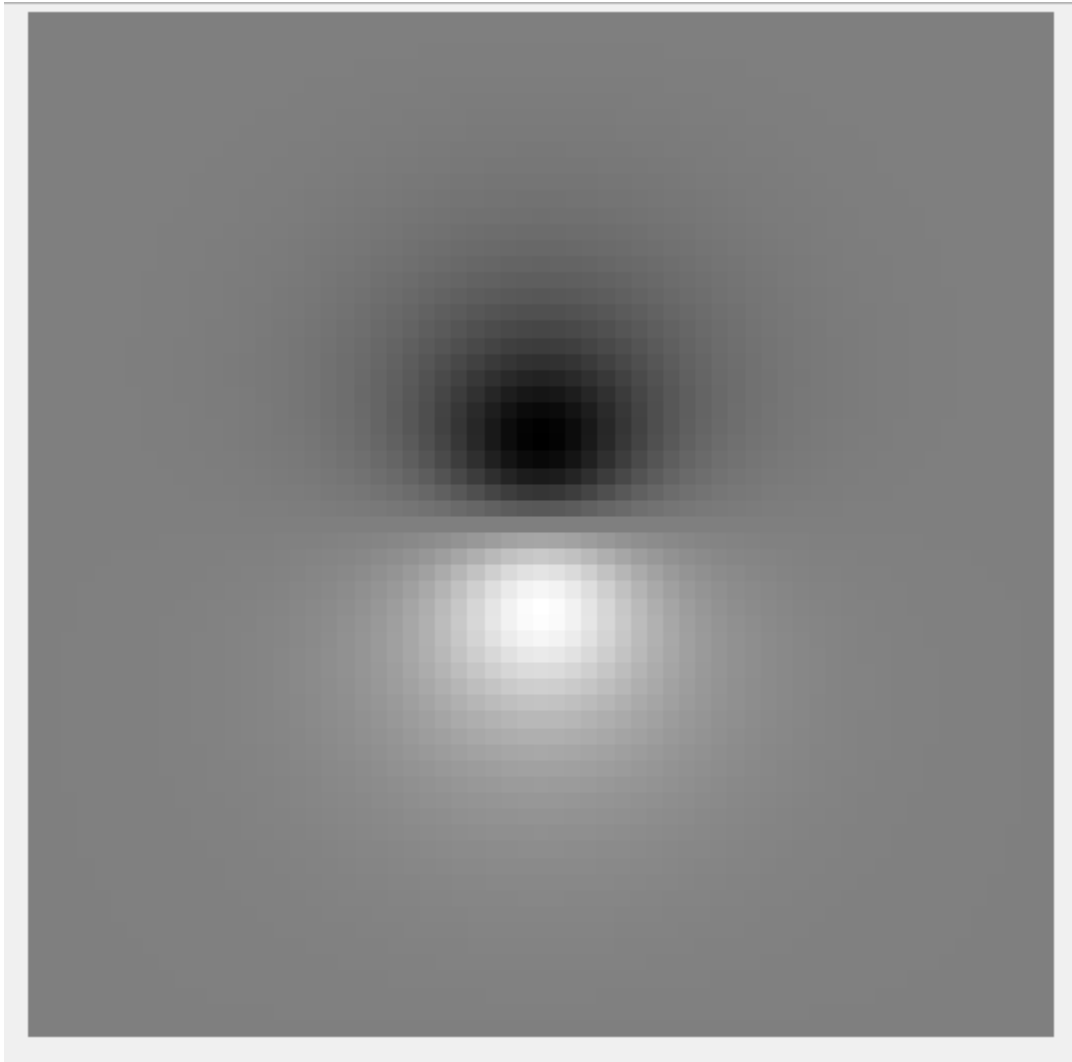
Pozostałe rozkłady mogą zostać odtworzone poprzez uruchomienie symulacji we własnym zakresie - patrz [literatura](#).

3.3 Losowa minimalizacja.

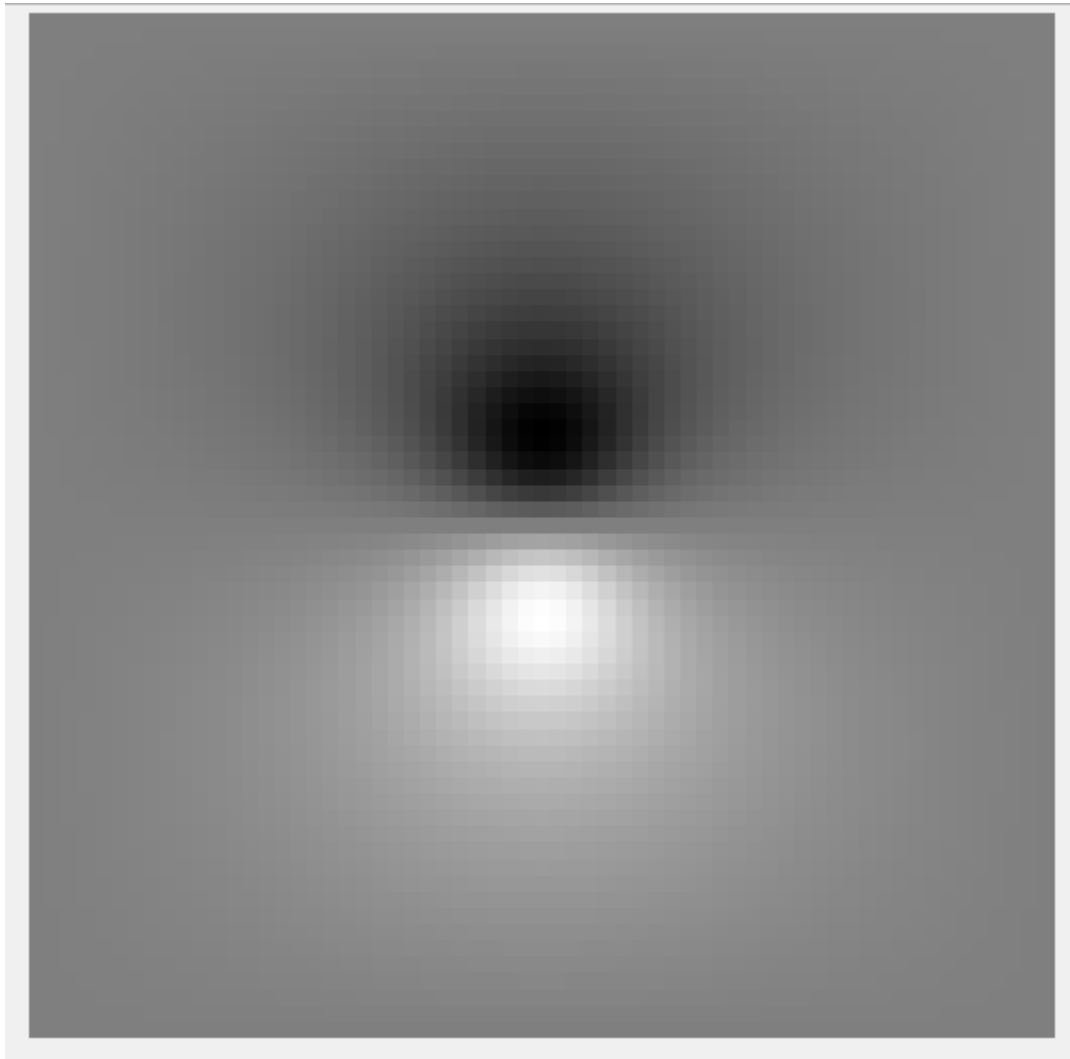
Wykonano również minimalizację losową. W tym celu, dla wartości $r = 0.1$ losowano pewną wartość liczbową z zakresu $u_0 \in \langle -r, r \rangle$ następnie sprawdzano, czy wartość S przed zmianą była większa niż po zmianie. Jeżeli tak, aplikowano zmianę.

Poniższy wykres przedstawia zależność wartości S od numeru iteracji.

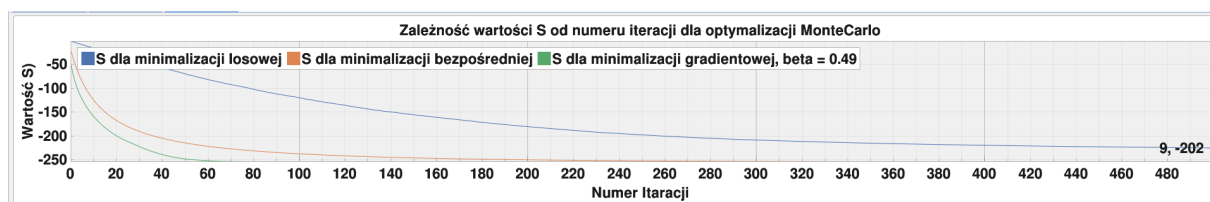
Wykonano kopię obrazu $u(i, j)$ co 100 iteracji. Postęp przedstawiono poniżej



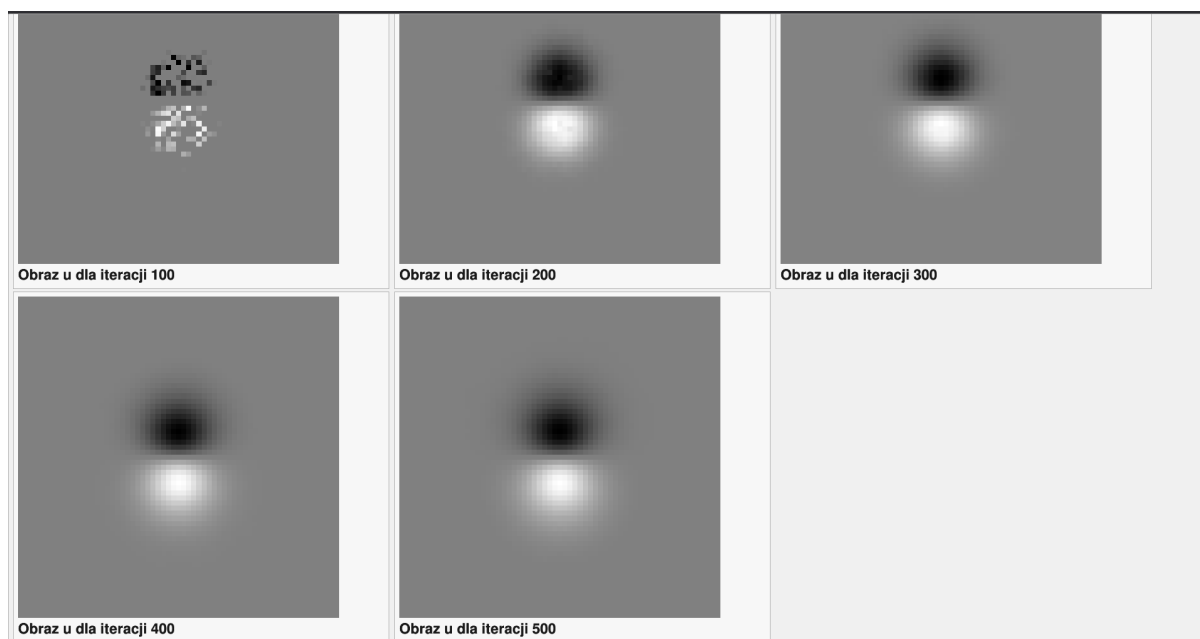
Rys. 3.4: Rozkład $u(i, j)$ dla $\beta = 0.1$



Rys. 3.5: Rozkład $u(i, j)$ dla $\beta = 0.49$



Rys. 3.6: Zależność wartości S od numeru iteracji



Rys. 3.7: Obrazy $u(i, j)$ dla poszczególnych iteracji.

3.4 Porównanie wydajności czasowej poszczególnych metod

Operacja	Czas wykonania [ms]
Minimalizacja bezpośrednia funkcjonału S	1.275s
Minimalizacja gradientowa, $\beta = 0.1$	0.4842s
Minimalizacja gradientowa, $\beta = 0.3$	0.4812s
Minimalizacja gradientowa, $\beta = 0.4$	0.4665s
Minimalizacja gradientowa, $\beta = 0.49$	0.5196s
Minimalizacja gradientowa (total)	1.952s
Minimalizacja losowa	0.5409s

4 Podsumowanie

Jak widać, najbardziej efektywne zarówno czasowo jak i pod względem szybkości minimalizacji S jest podejście minimalizacji gradientowej.

5 Literatura

- MOF5 - Program symulacyjny - <https://github.com/gucio321-studies/MOFProj5> rewizja b8f2c43d5b0a57494f49f2d1d1ffbffd0a84c6a7